

User Manual of the *C-Product Toolbox* *

Pablo Soto-Quiros[†] and Samuel Valverde-Sanchez[‡]

^{†,‡}*Escuela de Matemática, Instituto Tecnológico de Costa Rica, Cartago 30101, Costa Rica*

Emails: [†]jusoto@tec.ac.cr, and [‡]savalverde@itcr.ac.cr

ORCID: [†]<https://orcid.org/0000-0003-2903-3116>, and [‡]<https://orcid.org/0000-0003-2903-3116>

Abstract

The *C-Product Toolbox* is a computational package designed to operate with third-order tensors using the reduced *c*-product, an optimized version of the *c*-product based on the discrete cosine transform. Developed in MATLAB and PYTHON, this software improves computational efficiency by reducing memory usage and execution time, as it exclusively uses real arithmetic and avoids the complex calculations inherent to the *t*-product, which is based on the discrete Fourier transform. Furthermore, it incorporates advanced functionalities not available in previous tools, such as the tensor versions of the pseudoinverse and the Drazin inverse.

Contents

1	Introduction	1
2	Description and use of the functions in the <i>C-Product Toolbox</i>	3
3	Quick guide to install the <i>C-Product Toolbox</i> in Matlab	7
4	Quick guide to install the <i>C-Product Toolbox</i> in Python	9

1. Introduction

1.1. General description

The *C-Product Toolbox* is a computational package designed to perform operations on third-order tensors using a variant of the *c*-product [1] called the *reduced c-product* [2]. This toolbox is available for both MATLAB and PYTHON, facilitating its adoption by a wide community of researchers and professionals in scientific computing and signal processing.

The reduced *c*-product is based on the discrete cosine transform (DCT), which improves the computational efficiency of tensor operations compared to other approaches, such as the *t*-product [3], which uses the discrete Fourier transform (DFT) and requires complex arithmetic. A detailed explanation of the reduced *c*-product is presented in Section 1.4.

1.2. Advantages of the *C-Product Toolbox*

The *C-Product Toolbox* offers advanced features not available in other similar tools, including tensor pseudoinverse, tensor Drazin inverse, and tensor equation solving by least squares. All functions have been implemented in both MATLAB and PYTHON using the NUMPY and SCIPY libraries. Being implemented in PYTHON, it allows integration into open-source environments without relying exclusively on MATLAB. This toolbox is an optimized alternative to the *Tensor-Tensor Product Toolbox 2.0* [4], based on the *t*-product, which has higher computational resource consumption and lacks recent updates.

Thanks to its formulation based on the DCT, the *C-Product Toolbox* offers key advantages, such as **lower memory consumption and reduced execution time**. Additionally, the numerical experiments reported in [2] demonstrate that this toolbox significantly improves performance in terms of computational efficiency and memory usage. Its applicability has been validated in various signal and image processing tasks, with notable results in noise removal from grayscale videos.

1.3. GitHub repository of the *C-Product Toolbox*

The source code of the *C-Product Toolbox* is available at the following GitHub repository:

<https://github.com/jusotoTEC/c-product-toolbox>

Observación 1. *The GitHub repository of the C-Product Toolbox includes all the examples and numerical experiments presented in [2].*

1.4. Formal definition of the reduced *c*-product

In this section, we introduce some fundamental concepts and definitions related to the *reduced c-product*. A third-order tensor $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ is a three-dimensional structure that generalizes the notion of a matrix, where each element is given by $\mathcal{A}(i, j, k) \in \mathbb{R}$, for all $i = 1, \dots, m$, $j = 1, \dots, n$, and $k = 1, \dots, p$.

Definición 1 ([1]). *Let $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ and $\mathcal{B} \in \mathbb{R}^{n \times s \times p}$. We define the **face product** as the tensor $\mathcal{C} \in \mathbb{R}^{m \times s \times p}$ given by:*

$$\mathcal{C}^{(i)} = (\mathcal{A} \triangle \mathcal{B})^{(i)} = \mathcal{A}^{(i)} \mathcal{B}^{(i)},$$

where $\mathcal{A}^{(i)}$ and $\mathcal{B}^{(i)}$ represent the i -th frontal slices of the tensors $\mathcal{A}$ and $\mathcal{B}$, respectively, for all $i = 1, \dots, p$.

Definición 2 ([1], [5]). *The **mode-3 product** between a tensor $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ and a matrix $M \in \mathbb{R}^{q \times p}$ is the tensor $\mathcal{B} \in \mathbb{R}^{m \times n \times q}$ defined as:*

$$\mathcal{B} = \mathcal{A} \times_3 M,$$

where each element of $\mathcal{B}$ is obtained through the transformation:

$$\mathcal{B}(i, j, :) = \text{vec}^{-1}(M \text{vec}(\mathcal{A}(i, j, :))),$$

for all $i = 1, \dots, m$ and $j = 1, \dots, n$. Here, $\text{vec} : \mathbb{R}^{1 \times 1 \times p} \rightarrow \mathbb{R}^p$ is the operator that converts a tubular vector in the set $\mathbb{R}^{1 \times 1 \times p}$ into a column vector in the set $\mathbb{R}^p$, and $\text{vec}^{-1} : \mathbb{R}^p \rightarrow \mathbb{R}^{1 \times 1 \times p}$ is its inverse.

Definición 3 ([1]). *The **type-II DCT matrix** is defined as:*

$$C_p(i, j) = \sqrt{\frac{2}{p(1 + \delta(i, 1))}} \cos\left(\frac{\pi(2j - 1)(i - 1)}{2p}\right),$$

for $i, j = 1, 2, \dots, p$, where $\delta(i, 1)$ is the Kronecker delta function, given by:

$$\delta(i, j) = \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{if } i \neq j. \end{cases}$$

With these tools established, we now present the definition of the **reduced c-product**.

Definición 4 ([2]). *The **reduced c-product** between the tensors $\mathcal{A} \in \mathbb{R}^{m \times n \times p}$ and $\mathcal{B} \in \mathbb{R}^{n \times s \times p}$ is defined as:*

$$\mathcal{A} *_c \mathcal{B} = L^{-1}(L(\mathcal{A}) \triangle L(\mathcal{B})),$$

where the transformations L and L^{-1} are defined by $L(\mathcal{A}) = \mathcal{A} \times_3 C_p$ and $L^{-1}(\mathcal{A}) = \mathcal{A} \times_3 C_p^T$.

2. Description and use of the functions in the *C-Product Toolbox*

Table 1 presents a list of the functions implemented in the *C-Product Toolbox*, along with a brief description, their syntax, and the section where their use is detailed.

Function	Description	Syntax	Reference
<code>cprod</code>	Reduced <i>c</i> -product	<code>C = cprod(A,B)</code>	Section 2.1
<code>cinprod</code>	Tensor inner product	<code>c = cinprod(A,B)</code>	Section 2.2
<code>ceye</code>	Identity tensor	<code>I = ceye(n,p)</code>	Section 2.3
<code>ctransp</code>	Tensor transpose	<code>B = ctransp(A)</code>	Section 2.4
<code>csvd</code>	<i>c</i> -SVD	<code>[U,S,V] = csvd(A,opt)</code>	Section 2.5
<code>cqr</code>	<i>c</i> -QR	<code>[Q,R] = cqr(A,opt)</code>	Section 2.6
<code>csqrtt</code>	Tensor square root	<code>X = csqrtt(A)</code>	Section 2.7
<code>ctubalrank</code>	Tubal rank	<code>trank = ctubalrank(A)</code>	Section 2.8
<code>cmultirank</code>	Multi-rank	<code>mrnk = cmultirank(A)</code>	Section 2.9
<code>cinv</code>	<i>c</i> -inverse	<code>X = cinv(A)</code>	Section 2.10
<code>cpinv</code>	<i>c</i> -pseudoinverse	<code>X = cpinv(A)</code>	Section 2.11
<code>cdrazin</code>	<i>c</i> -Drazin inverse and multi-index	<code>[X,t] = cdrazin(A)</code>	Section 2.12
<code>cnorm</code>	Tensor norm	<code>z = cnorm(A,opt)</code>	Section 2.13
<code>clowrank</code>	Low tubal-rank tensor approximation problem	<code>Ar = clowrank(A,r)</code>	Section 2.14
<code>csvt</code>	Tensor singular value thresholding problem	<code>D = csvt(A,t)</code>	Section 2.15
<code>clsq</code>	Tensor least squares problem	<code>X = tlsq(A,B,C)</code>	Section 2.16
<code>test_toolbox</code>	Numerical examples	<code>test_toolbox()</code>	Section 2.17

Table 1. Functions available in the *C-Product Toolbox*.

Observación 2. For a detailed explanation of the mathematical foundations of each function in the *C-Product Toolbox* included in Table 1, it is recommended to consult Section 2 of [2].

Below, the help documentation corresponding to each function in the *C-Product Toolbox* is presented.

2.1. `cprod`

Description: Computes the reduced *c*-product between two tensors
Function syntax: `C = cprod(A, B)`
Input: `A` = tensor of dimension `m1 x n1 x p1`
`B` = tensor of dimension `m2 x n2 x p2`
Output: `C` = tensor of dimension `m1 x n2 x p1`

2.2. cinprod

Description: Calculates the inner product between two tensors
 Function syntax: `c = cinprod(A,B)`
 Input: `A` = tensor of dimension $m \times n \times p$
 `B` = tensor of dimension $n \times m \times p$
 Output: `c` = inner product between tensors `A` and `B`

2.3. ceye

Description: Computes the identity tensor under the reduced c-product
 Function syntax: `I = ceye(n,p)`
 Input: `n` = positive integer
 `p` = positive integer
 Output: `I` = tensor of dimension $n \times n \times p$

2.4. ctransp

Description: Computes the transpose of a tensor under the reduced c-product
 Function syntax: `B = ctransp(A)`
 Input: `A` = tensor of dimension $m \times n \times p$
 Output: `B` = tensor of dimension $n \times m \times p$

2.5. csvd

Description: Computes the SVD decomposition of a tensor under the reduced c-product.
 Function syntax: `[U, S, V] = cpinv(A, opt)`
 Input: `A` = tensor of size $m \times n \times p$
 `opt` = options for various results of `U`, `S`, and `V`
 1) 'full': (default) produces the full SVD decomposition of the tensor, i.e., $A = U*S*V^T$, where:
 `U` = tensor of size $m \times m \times p$
 `S` = tensor of size $m \times n \times p$
 `V` = tensor of size $n \times n \times p$
 2) 'econ': produces the 'economy-sized' decomposition. Let $s = \min(m,n)$. Then, $A = U*S*V^T$, where:
 `U` = tensor of size $m \times s \times p$
 `S` = tensor of size $s \times s \times p$
 `V` = tensor of size $n \times s \times p$
 Output: `U`, `S`, `V`

2.6. `cqr`

Description: Computes the tensor QR decomposition
under the reduced c-product

Function Syntax: `[Q,R] = cqr(A,opt)`

Input: `A` = tensor of dimension $m \times n \times p$

`opt` = options for various `Q` and `R` results

- 1) 'full': (default) produces the full
tensor QR decomposition, i.e., $A = Q \ast R$,
where:
 Q = tensor of dimension $m \times m \times p$
 R = tensor of dimension $m \times n \times p$
- 2) 'econ': produces the 'economy-sized' decomposition
when $m > n$. Let $A = Q \ast R$, where:
 Q = tensor of dimension $m \times n \times p$
 R = tensor of dimension $n \times n \times p$
 If $m \leq n$, then the 'economy-sized'
decomposition is the same as the
full decomposition

Output: `Q`, `R`

2.7. `csqrtt`

Description: Computes the principal square root of a tensor
under the reduced c-product

Function syntax: `X = csqrtt(A)`

Input: `A` = tensor of dimension $m \times m \times p$

Output: `X` = tensor of dimension $m \times m \times p$

2.8. `ctubalrank`

Description: Computes the tubal rank of a tensor using
the c-SVD method

Function syntax: `trank = ctubalrank(A)`

Inputs: `A` = tensor of dimension $m \times n \times p$

Output: `trank` = non-negative integer

2.9. `cmultirank`

Description: Computes the multi-rank of a tensor using
the reduced c-product

Function syntax: `mrnk = cmultirank(A)`

Inputs: `A` = tensor of dimension $m \times n \times p$

Output: `mrnk` = vector of dimension p

2.10. `cinv`

Description: Computes the tensor inverse
under the reduced c-product

Function syntax: `X = cinv(A)`

Inputs: `A` = tensor of dimension $m \times m \times p$

Output: `X` = tensor of dimension $m \times m \times p$

2.11. `cpinv`

Description: Computes the tensor pseudoinverse
under the reduced c-product

Function syntax: `X = cpinv(A)`

Inputs: `A` = tensor of dimension $m \times n \times p$

Output: `X` = tensor of dimension $n \times m \times p$

2.12. `cdrazin`

Description: Computes the Drazin inverse of a tensor and
the multi-index of a tensor under the c-product

Function syntax: `[X,t] = cdrazin(A)`

Inputs: `A` = tensor of dimension $m \times m \times p$

Outputs: `X` = tensor of dimension $m \times m \times p$

`t` = vector of dimension p

2.13. `cnorm`

Description: Computes the tensor norm under the c-product

Function syntax: `z = cnorm(A,opt)`

Inputs: `A` = tensor of dimension $m \times n \times p$

`opt` = options for different types of tensor norms:

1) 'fro' : Frobenius norm

2) 'spec': Spectral norm

3) 'nuc' : Nuclear norm

Output: `z` = non-negative number

2.14. `clowrank`

Description: Computes the solution to the low tubal-rank
tensor problem using the reduced c-product

Function syntax: `Ar = clowrank(A,r)`

Inputs: `A` = tensor of dimension $m \times n \times p$

`r` = positive integer such that $r \leq \min(m,n)$

Output: `Ar` = tensor of dimension $m \times n \times p$

2.15. csvt

Description: Computes the solution to the tensor singular value thresholding problem under the reduced c-product

Function syntax: `X = csvt(A,t)`

Inputs: `A` = tensor of dimension $m \times n \times p$
`t` = positive constant

Output: `X` = tensor of dimension $m \times n \times p$

2.16. clsq

Description: Computes the solution to the tensor least squares problem under the reduced c-product

Function syntax: `X = clsq(A,B,C)`

Inputs: `A` = tensor of dimension $m \times n \times p$

`B` = tensor of dimension $m \times r \times p$

`C` = tensor of dimension $s \times n \times p$

Output: `X` = tensor of dimension $r \times s \times p$

2.17. test_toolbox

Description: Numerical examples to evaluate the performance and functionality of the C-Product Toolbox

Function syntax: `test_toolbox()`

3. Quick guide to install the *C-Product Toolbox* in Matlab

3.1. Download and file location

- Download the folder `cproduct_matlab` from the GitHub repository located at <https://github.com/jusotoTEC/c-product-toolbox>
- The folder `cproduct_matlab` contains the MATLAB functions of the *C-Product Toolbox*.
- Ensure that the folder `cproduct_matlab` is located in an accessible directory.

3.2. Add the toolbox to the Matlab path

To make MATLAB recognize the *C-Product Toolbox*, follow these steps:

1. Open MATLAB.
2. In the command window, run the following command:

```
1 addpath('path/to/cproduct_matlab');
2 savepath;
```

Replace 'path/to/cproduct_matlab' with the actual location of the folder on your system.

3.3. Verify the installation

To check that the functions are available, run the following in the command line:

```
1 which function_name
```

For example:

```
1 which cprod
```

If the installation was successful, MATLAB will display the location of the corresponding file.

3.4. Using the *C-Product Toolbox* in Matlab

You can now invoke the functions from the *C-Product Toolbox* just like any other MATLAB function. Figure 1 shows examples of how to use all the available functions from the *C-Product Toolbox* within the MATLAB environment.

```
1 % Example of applying the C-Product Toolbox functions in MATLAB
2
3 A = rand(10,5,3);           % Third-order tensor generated ...
4                               % ... randomly with uniform distribution
5 B = rand(5,5,3);           % Third-order tensor generated ...
6                               % ... randomly with uniform distribution
7 C = cprod(A,B);             % c-product between A and B
8 c = cprod(A,B);             % Tensorial inner product between A and B
9 I = ceye(10,5);             % Identity tensor of size 10 x 10 x 5
10 At = ctransp(A);           % Tensorial transpose of A
11 [U,S,V] = csvd(A);          % c-SVD of A
12 [Q,R] = cqr(A);             % c-QR of A
13 X1 = cqr(A);                % Tensorial square root of A
14 trank = ctubalrank(A);      % Tubal rank of A
15 mrank = cmultitank(A);      % Multi-rank of A
16 X2 = cinv(B);               % c-inverse of B
17 X3 = cpinv(A);              % c-pseudoinverse of A
18 [X4,t] = cdrazin(A);        % c-Drazin inverse of A and multi-index
19 z1 = cnorm(A,'fro');         % Frobenius tensor norm of A
20 z2 = cnorm(A,'spec');        % Spectral tensor norm of A
21 z3 = cnorm(A,'nuc');         % Nuclear tensor norm of A
22 Ar = clowrank(A,3);         % Tensor solution for the low-tubal-rank ...
23                               % ... approximation problem
24 X5 = csvt(A,0.5);           % Tensor solution for the tensor singular value ...
25                               % ... thresholding problem with tao=0.5
26 A1 = rand(5,4,3);           % Third-order tensor generated ...
27                               % ... randomly with uniform distribution
28 B1 = rand(5,2,3);           % Third-order tensor generated ...
29                               % ... randomly with uniform distribution
30 C1 = rand(2,4,3);           % Third-order tensor generated ...
31                               % ... randomly with uniform distribution
32 X6 = clsq(A1,B1,C1);        % Tensor solution for the tensor least squares problem
```

Figure 1. Numerical Example of the *C-Product Toolbox* in MATLAB

3.5. (Optional) Automatically load the *C-Product Toolbox* at Matlab startup

If you want the toolbox to load automatically each time you start MATLAB, add the line `addpath('path/to/cproduct_matlab');` to the `startup.m` file. To do so:

1. In MATLAB, run the command:

```
1 edit startup
```

2. Add the line:

```
1 addpath('path/to/cproduct_matlab');
```

3. Save and close the file.

Following these steps, the *C-Product Toolbox* will be ready to use in MATLAB.

4. Quick guide to install the *C-Product Toolbox* in Python

4.1. Prerequisites

Before installing the *C-Product Toolbox*, make sure you have the following:

- **Python 3.12** (It is recommended to use the latest stable version).
- **Required libraries:** You will need NumPy and SciPy. To install them, run:

```
1 pip install numpy
2 pip install scipy
```

4.2. Download and file location

- Download the `cproduct_python` folder from the GitHub repository at <https://github.com/jusotoTEC/c-product-toolbox>
- The `cproduct_python` folder contains the Python functions of the *C-Product Toolbox*.
- Ensure that you have the `cproduct_python` folder containing the following files:
 - `cproduct.py`: Implementation of the functions
 - `test_toolbox.py`: Example usage

4.3. Adding the *C-Product Toolbox* to the Python environment

If you want to import the functions from anywhere, add the folder path to `sys.path` in your code:

```
1 import sys
2 sys.path.append("path/to/cproduct_python") # Replace with the actual location
```

4.4. Verify the installation

Open a terminal or Python console and run:

```
1 import cproduct
2 print("C-Product Toolbox successfully imported.")
```

If no errors occur, the installation was successful.

4.5. Using the *C-Product Toolbox* in Python

To use the functions of the *C-Product Toolbox*, import the module into your script:

```
1 import cproduct as cp
```

Figure 2 shows the usage of all the functions of the *C-Product Toolbox* in PYTHON.

```
1 # Example application of the C-Product Toolbox functions in Python
2
3 import cproduct as cp          # Import the C-Product Toolbox
4 import numpy as np            # Import NumPy
5
6 A = np.random.rand(10,5,3)    # Third-order tensor generated ...
7                                # ... randomly with uniform distribution
8 B = np.random.rand(5,5,3)     # Third-order tensor generated ...
9                                # ... randomly with uniform distribution
10 C = cp.cprod(A,B);            # c-product between A and B
11 c = cp.cprod(A,B)             # Tensor inner product between A and B
12 I = cp.ceye(10,5)            # Identity tensor of size 10 x 10 x 5
13 At = cp.ctransp(A)            # Tensor transpose of A
14 [U,S,V] = cp.csvd(A)          # c-SVD of A
15 [Q,R] = cp.cqr(A)             # c-QR of A
16 X1 = cp.cqr(A)                # Tensor square root of A
17 trank = cp.ctubalrank(A)      # Tubal rank of A
18 mrank = cp.cmultitank(A)     # Multi-rank of A
19 X2 = cp.cinv(B)               # c-inverse of B
20 X3 = cp.cpinv(A)              # c-pseudoinverse of A
21 [X4,t] = cp.cdrazin(A)        # c-Drazin inverse of A and multi-index
22 z1 = cp.cnorm(A,'fro')        # Frobenius tensor norm of A
23 z2 = cp.cnorm(A,'spec')       # Spectral tensor norm of A
24 z3 = cp.cnorm(A,'nuc')        # Nuclear tensor norm of A
25 Ar = cp.clowrank(A,3)         # Tensor solution of the ...
26                                # ... tensor approximation problem with tubal rank 3
27 X5 = cp.csvt(A,0.5)           # Tensor solution of the ...
28                                # ... tensor singular value thresholding problem ...
29                                # ... with tau=0.5
30 A1 = np.random.rand(5,4,3)    # Third-order tensor generated ...
31                                # ... randomly with uniform distribution
32 B1 = np.random.rand(5,2,3)    # Third-order tensor generated ...
33                                # ... randomly with uniform distribution
34 C1 = np.random.rand(2,4,3)    # Third-order tensor generated ...
35                                # ... randomly with uniform distribution
36 X6 = cp.clsq(A1,B1,C1)        # Tensor solution of the tensor least squares problem
```

Figure 2. Numerical example of the *C-Product Toolbox* in PYTHON

■ References

- [1] E. Kernfeld, M. Kilmer, and S. Aeron, “Tensor–tensor products with invertible linear transforms”, *Linear Algebra and its Applications*, vol. 485, pp. 545–570, 2015.
- [2] P. Soto-Quiros, “*C-product toolbox*: A computational package for third-order tensor operations based on the reduced *c*-product”, *Journal of Computational Mathematics and Data Science*, 2025, Under review.
- [3] M. E. Kilmer and C. D. Martin, “Factorization strategies for third-order tensors”, *Linear Algebra and its Applications*, vol. 435, no. 3, pp. 641–658, 2011.
- [4] C. Lu, *Tensor-tensor product toolbox*, <https://github.com/canyilu/tproduct>, Carnegie Mellon University, Jun. 2018.
- [5] H. Jin, S. Xu, Y. Wang, and X. Liu, “The moore–penrose inverse of tensors via the m-product”, *Computational and Applied Mathematics*, vol. 42, no. 6, p. 294, 2023.